

Quality on the Money Path

Building a QA function for a payments platform moving 100,000 transactions a day — plus a working test plan and automation suite, a mentoring plan, a code review, and a risk-based strategy for multi-currency FX settlement. *Every claim here was executed, not asserted.*

PLAYWRIGHT	KARATE	FOUND	TARGET
7 tests passing	23 scenarios passing	11 defects & observations	Book Store demoqa.com

PART 1.1

Test Strategy & Design

The situation in one line: 100,000 transactions a day move through Go services with no automated tests, deployed daily by five engineers, with two manual QAs and frequent production bugs.

The instinct is to start writing tests. That is the wrong first move. With daily deploys and no safety net, the team is already shipping defects at a known rate, and I do not yet know *which* defects, *where* they come from, or *which* would hurt most. Ten days of measurement makes the next fifty days aim at something real.

One framing I will keep coming back to: in payments, a bug is not a bad user experience. It is somebody's money in the wrong place. A double-charged customer, a refund that silently fails, a settlement that is out by two units of currency — those are the defects that cost a payments company its licence to operate, and they are not the ones a login test finds.

Q1. How would you build the QA function? — the 30-day plan

Days 1–10: find out where the bleeding is

ACTIVITY	OUTPUT
Read every production incident and bug ticket from the last 90 days. Classify by flow, root cause, severity, and <i>the stage that should have caught it</i> .	An escape analysis: "68% of our escapes are refund edge cases that a service-level test would have caught." This one document justifies everything that follows.
Map the money path with the three backend developers: transaction → capture → refund → settlement. Every state, every external call, every place money can be double-counted or lost.	A risk map, ranked. This drives all coverage decisions from here on.
Sit with the two junior QAs and watch them test a release, without intervening.	Where the manual effort actually goes, and how much of it is repetitive regression that automation should own.
Trace one release end-to-end. Who decides it ships? What is checked? What is not?	The current release process, written down — usually the first time anyone has seen it whole.
Talk to support and finance/ops.	The defects customers <i>feel</i> , which are often not the ones engineering tracks.

Deliverable at day 10: a one-page risk and escape analysis presented to the team, with a proposed top-five critical flows. Not a QA manifesto — evidence, and a proposal.

Days 11–20: stop the bleeding, then automate the highest-value path

Two things run in parallel, because automation takes weeks and production bugs are happening today.

Immediate (manual, day 11): a critical-path release checklist for the top five flows — payment authorisation, capture, full and partial refund, merchant onboarding, settlement run. The juniors execute it before each deploy. It is not elegant and it does not scale, but from day 11 nothing ships without someone having checked the money path. It buys the runway to build the real thing.

Foundation (days 11–20):

PRIORITY	WHAT	WHY FIRST
1	API/integration tests for the Go transaction and refund services , running in CI	This is where money moves. It is also the fastest, most stable layer: a payment service test runs in milliseconds and does not break because a button moved.
2	Test environment and deterministic test data — docker-compose stack, a PSP sandbox or mock, seeded merchants, a card-scenario matrix	Nothing else is possible without this. Most failing QA functions fail here, not on tooling.
3	CI gate : unit → service tests → smoke, blocking merge	An automated test that does not block a merge is documentation, not a gate.
4	Bug workflow : severity matrix, reproduction template, triage within one working day, root-cause tagging	Makes escape analysis a continuous measurement instead of a one-off exercise.

Days 21–30: extend to the front end and hand ownership over

ACTIVITY	OUTPUT
Playwright suite for checkout and the merchant dashboard — the two or three journeys that matter, not everything	A UI regression gate on the flows that earn revenue
Pair both junior QAs into the automation, one flow each, with a code review from me	They stop being "manual only" — the single biggest capacity increase available to this team
QA joins refinement; testability and acceptance criteria become part of Definition of Ready	Defects prevented, not found. Cheapest quality there is.
Post-deploy smoke against production with a synthetic merchant	Detection in minutes rather than by customer complaint
Dashboard: escape rate, change failure rate, suite runtime, flake rate	Makes the QA function's impact arguable with numbers

Deliverable at day 30: the money path is covered by automated tests running on every merge; the release process is written down and enforced; both juniors have merged

automation code; there is a baseline measurement of quality to improve against.

What I would deliberately *not* do in the first 30 days

- **Not build a big UI regression suite.** It is the slowest, most brittle layer, and with 100K transactions a day the risk lives in the backend.
- **Not introduce a heavyweight test management tool.** Tests live in the repo next to the code. A tool becomes worth it once there is something to manage.
- **Not push for a separate QA phase or a staging freeze.** That fights daily deploys instead of supporting them.
- **Not chase a coverage percentage.** 80% line coverage of a payment service that does not test the double-refund case is worse than useless: it is false confidence.

Q2. What processes and tools would I introduce?

Processes

Shift left — QA present at refinement. For each story we answer three questions before a line is written: *how would we know this is broken? what happens on a partial failure? what would make this untestable?* Fifteen minutes here removes days of back-and-forth later. In payments, "what happens if the PSP times out after debiting but before responding?" is a refinement question, not a test-execution question.

Definition of Ready / Definition of Done. Ready: acceptance criteria exist, edge cases are named, test data is available. Done: automated tests written and passing, observability in place (can we see this fail in production?), no known Critical or High defects. QA does not "sign off" work at the end; QA makes it impossible to call something done without evidence.

Risk-based testing, always. We will never test everything, so we choose openly: critical money paths get exhaustive automated coverage; supporting features get happy path plus key negatives; cosmetic changes get exploratory review. That decision is written on the story, so it can be challenged.

Bug triage with root-cause tagging. Every defect gets a severity (customer impact), a priority (when we fix it), and a "should have been caught at" tag. That last one turns individual bugs into a signal about where the process leaks.

Blameless incident review with a test as the output. Every production defect ends with a regression test that would have caught it. This is the mechanism by which the suite becomes shaped like the product's actual failure modes rather than someone's guess.

QA embedded, not a gate at the end. QA sits in standups, reviews pull requests, and pairs with developers. A release captain rotates weekly, including the developers — the person who ships is the person who watches the dashboards.

Testing workflow

Developer commits

- └ pre-commit: lint, unit tests (seconds)
- └ PR: unit + service/integration + contract tests + smoke E2E ← blocks merge (< 10 min)
- └ merge to main: full API regression + deploy to staging
- └ staging: full E2E regression, nightly cross-browser ← blocks release
- └ production deploy: post-deploy smoke on a synthetic merchant ← auto-rollback trigger
 - + reconciliation and ledger monitors

The constraint that matters: **the PR gate must stay under ten minutes**. Slower than that and developers batch changes, which is exactly what daily deployment is meant to prevent. Everything that cannot fit goes nightly.

Environments

ENVIRONMENT	PURPOSE	DATA	PSP / EXTERNAL
Local	Developer feedback in seconds	docker-compose, seeded fixtures	Mocked, deterministic
CI (ephemeral)	The merge gate	Generated per run, torn down after	Mocked, with failure injection
Staging	Release candidate, integration truth	Anonymised production-shaped data, refreshed nightly	PSP sandbox — real protocol, fake money
Production	Post-deploy confidence	Synthetic merchant, small real amounts	Real, monitored and reconciled

Test data is the part that decides whether any of this works. For payments that means: a card-scenario matrix (approve, decline, insufficient funds, 3DS challenge, timeout, partial capture), merchant fixtures per configuration, and idempotency keys generated per run. Built by factories in code, never by hand-maintained SQL dumps that drift.

Tools

LAYER	TOOL	WHY THIS ONE
Go unit + service tests	Standard <code>testing</code> , <code>testify</code> , <code>httptest</code>	Native, fast, already familiar to the backend team — adoption is the constraint, not features
Go integration with real dependencies	<code>testcontainers-go</code>	Real Postgres and Kafka in CI; the difference between "our mock agrees with us" and "the database agrees with us"
Contract testing between services	Pact, or OpenAPI schema validation if the team prefers lighter	Seven services deploying independently — this is where integration breaks silently
API automation	Karate	Readable by non-Java people, excellent JSON assertions, parallel by default (used in this assessment)
Web E2E	Playwright + TypeScript	Auto-waiting kills the flake class that plagues Selenium suites; traces make CI failures diagnosable without reproducing locally
Performance	k6	Scriptable in JS, CI-friendly; 100K/day has peaks that matter
Security	<code>gosec</code> , <code>govulncheck</code> , dependency scanning in CI; annual external pentest	PCI-DSS expects demonstrable practice, not a one-off
CI/CD	GitHub Actions (or the incumbent)	Fits where the code already lives
Observability	The team's existing stack — Grafana, Sentry — plus reconciliation alerts	Production is a test environment with real users; instrument it accordingly

I would deliberately choose tools the *developers* will use. A QA-only toolchain recreates the wall between QA and engineering that this plan exists to remove.

How QA integrates with the dev team

- **Ownership:** developers own unit and service tests. QA owns API contract suites, E2E, exploratory testing, and the health of the pipeline. QA reviews test code in pull requests the same way developers review production code.
- **The rule I would hold:** QA does not become the team's bottleneck or its safety blanket. If the only thing stopping a bad deploy is a person clicking through a checklist, the system is broken, not the person.
- **The two juniors:** their manual expertise is a genuine asset — they know where this product breaks. The goal is not to convert them into developers; it is to give them enough automation skill to encode what they already know, while keeping the exploratory testing that automation can never replace.

Q3. How would I measure success?

Metrics have to answer the question an executive actually asks: *are we shipping faster with fewer things going wrong?* Anything that only measures QA activity is noise.

The four that matter most

METRIC	DEFINITION	WHY IT IS THE REAL SIGNAL	90-DAY TARGET
Escaped defect rate	Production defects per release, weighted by severity	The honest measure of whether testing works	Down 60% from the day-10 baseline
Change failure rate	Deploys causing an incident or rollback	Connects quality directly to delivery (DORA)	Under 15%
Mean time to detect	Deploy → we know something is wrong	In payments, an hour of undetected breakage is an hour of misplaced money	Under 15 minutes for critical flows
Critical-path automated coverage	Of the ranked money-path risks, how many have an automated test	Coverage of what matters, not of what is easy	100% of P0 by day 60

Supporting health metrics

METRIC	TARGET	WHAT IT PROTECTS
Defect detection percentage by stage	> 85% caught before production	Tells us <i>where</i> to invest next
Test suite runtime (PR gate)	< 10 minutes	Developer trust; batching behaviour
Flake rate	< 2%	A suite people ignore is worse than no suite
Mean time to restore	< 1 hour	Recovery matters as much as prevention
Defect reopen rate	< 10%	Quality of the fix and of the bug report
Escaped defects by root cause	Trending, not targeted	Feeds the escape analysis loop

Payments-specific quality metrics — the ones finance cares about

These are the numbers I would put on a dashboard next to the engineering ones, because in this domain they *are* the definition of quality:

- **Settlement reconciliation breaks** per day (target: zero, alarmed immediately)
- **Duplicate charge incidents** (target: zero — a single occurrence is an incident review)
- **Refund failure rate** and mean age of stuck refunds
- **Ledger imbalance** detected by continuous invariant checks

Metrics I would refuse to report

- **Number of test cases written or executed.** Rewards volume, punishes deletion of dead tests.
- **Bugs found per tester.** Rewards logging trivia and creates an adversarial relationship with developers — precisely the dynamic the mentoring section addresses.
- **Percentage of test cases automated.** The denominator is arbitrary; 200 automated cosmetic checks beat two automated refund tests on this metric and lose in reality.
- **Code coverage as a target.** Useful as a diagnostic, corrosive as a goal.

How I would report it

A single dashboard, reviewed in a 20-minute monthly quality review with engineering and product. Each month answers three questions: what escaped and why, what we changed in response, and what the next biggest risk is. Trends, not snapshots — and honesty when a number gets worse, since a metric that only ever improves is a metric nobody trusts.

Test Planning & Automation

Both deliverables are in this repository.

Task A — Test plan: Appendix A. Risk-driven, with a coverage matrix for both layers, test data and environment strategy, and entry/exit criteria.

Task B — Automation:

	LOCATION	RESULT
Web (Playwright + TypeScript)	<code>playwright/</code>	7 tests passing against the live site
API (Karate)	<code>karate/</code>	23 scenarios passing against the live API

Both suites automate the required flow — register, login, search and add a book, view the collection, delete the book, log out. Both were executed against demoqa.com while writing this; the design decisions behind them are in the READMEs.

Eleven defects and observations came out of building it, documented with reproduction steps in Appendix B. Two are worth surfacing here, because they are the kind that a suite written to pass would never have found:

1. **Failed authentication returns 200 OK** with `{"status":"Failed"}`. Any client checking the status code — which is every generated client — treats a rejected credential as a success.
2. **Generating a token silently invalidates the existing session.** The API supports one active token per user, which is documented nowhere. A user logged in on a phone and a laptop loses one of them without explanation.

One positive result is worth recording too: the API correctly enforces object-level authorisation. The `userId` travels in the request body, which invites a naive implementation to trust it — this one does not, and the `@security` scenarios prove it.

Mentoring

The four problems, and what each one actually is

Before the plan, a diagnosis — because the four symptoms in the brief have different causes and the same fix would not work on all of them.

SYMPTOM	WHAT IT USUALLY REALLY IS
Struggles with programming concepts	Being asked to write code from a blank page before ever having read enough of it. Almost nobody learns to write in a language they have not read.
Test cases too vague, miss edge cases	Not a writing problem. A <i>thinking</i> problem: no systematic technique for generating cases, so they rely on imagination and imagination is uneven.
Discouraged when bugs are rejected	Partly report quality, partly that nobody has told them rejection is normal. A rejected bug feels like being told you are bad at your job.
Poor estimates	Estimating work they have not decomposed. Nobody can estimate "test the refund feature"; anyone can estimate "write six API cases for partial refunds".

Only one of these is a technical skill gap. That shapes everything below.

A. Three-month development plan

The goal by day 90: independently automates a small feature end to end, writes test cases a peer would not need to rewrite, and reports bugs that get accepted first time.

Month 1 — Foundations and confidence

Theme: read before you write.

WEEK	FOCUS	CONCRETE WORK
1	JavaScript fundamentals — variables, functions, async/await, arrays	30 min/day on exercism.io JS track. Pair with me for one hour; I write, narrating every decision.
2	Reading tests	Given our existing suite: explain what five tests do, in writing. Then <i>modify</i> three of them (change an assertion, add a case to a data table). No blank pages this month.
3	Test design technique	Equivalence partitioning and boundary value analysis, applied to our real password and amount validation. Deliverable: a set of cases derived by technique, not by intuition.
4	First contribution	Add two new cases to an existing suite by copying the shape of neighbouring ones. Full code review from me. Merged by end of month 1 — this matters psychologically far more than its technical value.

Success at month 1: explains async/await in their own words; has merged test code; writes cases with explicit boundaries.

Month 2 — Applied automation

Theme: write it yourself, with a net.

WEEK	FOCUS	CONCRETE WORK
5	Page object model	Refactor one existing spec's selectors into a page object. Refactoring teaches structure faster than greenfield code.
6	API testing	Automate three endpoints, including negative cases. API tests are the gentler on-ramp: no selectors, no waiting, clear pass/fail.
7	A whole small feature	Own the automation for one low-risk feature, from test design through to merged PR. I review, I do not write.
8	Debugging and flake	Give them a deliberately flaky test to diagnose. Teach traces, retries, and why <code>waitForTimeout</code> is the enemy.

Success at month 2: writes a new test from scratch without a template; can explain why a test failed rather than just re-running it.

Month 3 — Ownership and judgement

Theme: decide, don't just execute.

WEEK	FOCUS	CONCRETE WORK
9	Risk-based thinking	For an upcoming feature, they produce the test plan. We compare it against mine and discuss the differences — not the "gaps", the <i>differences</i> .
10	Code review	They review someone else's test PR. Reviewing is how you learn what good looks like.
11	Bug advocacy	Own triage for a week: write, defend, and follow up on every bug they raise.
12	Teach it back	A 30-minute session to the team on something they learned. Teaching exposes what is still shaky, and rebuilds standing with the developers.

Success at month 3: owns a feature area's testing; their bugs are accepted first time; their estimates land within $\pm 25\%$.

Targeted interventions for each specific problem

Programming struggle → **read, modify, then write.** Nobody learns to write code from a blank page. Month 1 is deliberately all reading and modification. Plus 20 minutes of pairing daily for the first two weeks, where *they* type and I talk — the typing is what builds the muscle memory, and the narration is what builds the model.

Vague test cases → **give them a generator, not feedback.** "Be more specific" is useless advice. A technique is not. I teach three: boundary value analysis, equivalence partitioning, and a checklist heuristic (empty, null, zero, negative, maximum, duplicate, concurrent, wrong type, unauthorised). Then a drill: take one existing vague case and expand it into eight specific ones. The rule I would give them — *a test case is specific enough when someone who has never seen the feature can execute it and get the same result* — turns a matter of taste into a testable standard.

Rejected bugs → **separate the report from the finding.** First, the reframe, said out loud: a rejected bug is not a failure, it is information, and every experienced tester has a pile of them. Then the craft. A bug report template — environment, steps, expected, actual, evidence, impact in business terms — plus one habit that changes the numbers fast: **spend five minutes with the developer before filing anything ambiguous.** It converts a rejection into a conversation. And we track their acceptance rate *together*, as a skill to improve, so the metric belongs to them rather than being used on them.

Estimation → decompose, then look back. Two rules. Nothing is estimated as a single unit larger than one day; anything bigger gets broken down first. And they keep a private log of estimate versus actual, reviewed in our 1:1 every fortnight. Nobody improves at estimating without seeing their own history — it is the only feedback loop that works. Expected: $\pm 50\%$ variance in month 1, $\pm 25\%$ by month 3. I would say those numbers out loud so that being wrong early is *expected*, not a failure.

B. How I would structure 1:1s

Weekly, 45 minutes, in their calendar, never cancelled. Cancelling a junior's 1:1 tells them their development is the first thing to go when I get busy.

TIME	SEGMENT	PURPOSE
5 min	How are you, actually	Not a formality. Discouragement shows up here before it shows up in the work.
10 min	Their agenda	They bring it. Blockers, questions, things they want to try.
15 min	Skill focus	The week's topic — pair on code, review a test design, walk through a rejected bug together.
10 min	Feedback, both directions	Specific praise for something concrete, one improvement, and <i>"what should I be doing differently as your manager?"</i>
5 min	Next week's commitment	One or two specific, achievable actions, written down.

This is not a status meeting. Status happens in standup. If a 1:1 turns into "what did you finish", the development conversation has been quietly cancelled.

Monthly, the shape changes: review the skills matrix, look at the estimate log together, and re-agree the next month's goals.

C. Specific resources and exercises

Reading (short and practical — a 700-page book will not get finished):

- *JavaScript.info* — sections on functions, arrays and promises only. Assigned as specific sections, never "read the site".
- Playwright's own documentation, particularly locators and auto-waiting. It is unusually good and it teaches the mental model, not just the API.

- Elisabeth Hendrickson, *Explore It!* — the best book on generating test ideas systematically. Directly targets the vague-test-cases problem.
- *Lessons Learned in Software Testing* (Kaner, Bach, Pettichord) — read as one lesson per week, discussed in the 1:1.

Exercises, each aimed at a named weakness:

EXERCISE	TARGETS
Exercism JavaScript track, 3 problems/week	Programming fluency
"Explain this test to me in writing" — five existing tests	Reading comprehension before authoring
"Test the login form" → then compare against my list of 20 cases	Edge case generation
Boundary drill: take a currency amount field, enumerate every boundary	Systematic thinking
Bug report rewrite: take three of their rejected bugs and rewrite them together	Bug advocacy
Break a working test deliberately, then fix it	Debugging confidence
Estimate a task, then measure, then explain the difference	Estimation
Review a PR of mine — I plant two real problems in it	Judgement, and permission to challenge me

That last one matters more than it looks. A junior who has once told their senior "this test is wrong" and been thanked for it behaves differently forever afterwards.

D. How I would measure progress

A skills matrix, scored together, monthly. Levels: 1 — needs help, 2 — with guidance, 3 — independent, 4 — teaches others.

SKILL	NOW	M1	M2	M3
JavaScript fundamentals	1	2	2	3
Reads and modifies existing tests	1	3	3	4
Writes new automated tests	1	1	2	3
Test case design and edge cases	2	2	3	3
Debugging failures	1	1	2	3
Bug reporting and advocacy	2	3	3	4
Estimation accuracy	1	2	2	3
Risk-based judgement	1	1	2	2

Objective evidence behind the scores, so this is not a matter of my impression:

- Test code merged (count and, more importantly, review comments per PR — trending down)
- Bug acceptance rate (baseline it in week 1, review monthly)
- Estimate variance (from their own log)
- Review turnaround: how much of my review is corrections versus discussion

The honest part. If by month 2 the automation is genuinely not landing, we have a frank conversation about direction — not a failure conversation. Exceptional exploratory and domain testers are rarer and more valuable than mediocre automation engineers, and someone who knows where a payments product breaks has a career that does not require writing Playwright. Steering someone toward what they are excellent at is mentoring; insisting they become a version of me is not.

Code Review Exercise

A note on how I would deliver this. In writing, on the PR, I would lead with what works and keep the tone collaborative — as below. But I would also spend twenty minutes pairing on it, because a review this long lands very differently in a conversation than as a wall of comments. The rewritten version is offered as a suggestion, not a patch to apply silently.

The review

Nice work getting a browser driving the app end to end — that is the hard first step, and the structure is already right: navigate, fill, submit, verify. Everything below is refinement of something that already works.

There is one issue that matters far more than the rest, so let me start there.

● Critical — this test cannot fail

```
if (url.includes('dashboard')) {  
  console.log('TEST PASSED');  
} else {  
  console.log('TEST FAILED');  
}
```

`console.log` prints text. It does not tell the test runner anything. When login breaks, this test prints "TEST FAILED" — and then reports **passed**, in green, and CI merges the change.

Why this matters more than any other point: a test that cannot fail is worse than no test at all. No test is an honest gap. This is a false signal, and the team will trust it right up until the day it lets a real login bug into production.

The fix: use assertions. An assertion throws on failure, and throwing is how a test communicates.

```
await expect(page).toHaveURL(/dashboard/);
```

If you take one thing from this review: **every test needs at least one assertion, and `console.log` is never one.** The quick self-check I use — *"if the feature were completely broken, would this test go red?"* If the answer is no, it is not a test yet.

● Critical — the import is wrong, so this never runs

```
const test = require('playwright'); // this package has no `test` export
```

`playwright` is the browser automation library. `@playwright/test` is the test runner that provides `test`, `expect`, `fixtures`, `reporting` and `retries`.

```
const { test, expect } = require('@playwright/test');
```

Worth knowing because the two packages are easy to confuse, and the error you get is not obvious.

● Important — `waitForTimeout(5000)` will make this slow *and* flaky

```
await page.waitForTimeout(5000);
```

A fixed sleep is a bet on how long something takes. You lose the bet in both directions: if login takes 200 ms, five seconds are wasted on every run; if the CI machine is loaded and login takes 5.5 seconds, the test fails for no reason. Multiply by a few hundred tests and this single habit is the difference between a four-minute suite and a forty-minute one.

Playwright's assertions retry automatically until they pass or time out, so waiting for the *condition* is both faster and more reliable:

```
await expect(page).toHaveURL(/dashboard/); // waits only as long as it needs to
```

Rule of thumb: **wait for a condition, never for a duration.**

● Important — asserting on the URL alone is a weak oracle

`url.includes('dashboard')` passes if the app redirects to `/dashboard` and then renders an error, or an empty shell, or a spinner that never resolves. It answers "did we navigate?" when

the question is "am I logged in?"

Assert on something only a logged-in user can see:

```
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
await expect(page.getByTestId('user-menu')).toContainText('test@test.com');
```

Also — `includes('dashboard')` would match `/login?error=dashboard-unavailable`. Prefer a precise pattern.

● Important — manual browser lifecycle leaks processes

```
const browser = await test.chromium.launch();
// ...
await browser.close();
```

If anything above `close()` throws, the browser is never closed. In CI, a few of those and the machine runs out of memory — usually surfacing as *other* tests failing mysteriously, which is a horrible thing to debug.

Playwright's `page` fixture handles this: a fresh isolated context per test, always cleaned up, even on failure. It also gives you traces and screenshots for free.

```
test('...', async ({ page }) => { /* browser lifecycle handled for you */ });
```

● Worth fixing — hardcoded URL and credentials

```
await page.goto('http://localhost:3000/login');
await page.fill('#email', 'test@test.com');
await page.fill('#password', 'password123');
```

The URL means this test only runs on your machine — not against staging, not in CI. Put it in `playwright.config.ts` as `baseUrl` and navigate with `page.goto('/login')`.

Credentials in the source have a second problem beyond configuration: the moment a real credential is committed, it is in the git history forever. The habit is worth building now, on a

fake password, so it is automatic later. Use environment variables, or better, create the user in setup so the test does not depend on data someone else might delete.

● Worth fixing — the test name does not say what it checks

```
// test for login
test('login test', async () => {
```

When this fails in CI at 2 a.m., the report says `login test failed` — which tells you nothing. A name that states the expected behaviour makes the report readable on its own:

```
test('logs a registered user in and lands them on the dashboard', ...)
```

The comment `// test for login` repeats the name; delete it. Good comments explain *why*, not *what* — the code already says what.

● Worth fixing — selectors tied to implementation

`#email`, `#password` and `#submit` are fine while those ids exist, but they break the moment someone refactors the markup, and they do not describe what a user sees. Playwright prefers user-facing locators:

```
await page.getByLabel('Email').fill(...);
await page.getByRole('button', { name: 'Sign in' }).click();
```

These survive refactoring and double as an accessibility check — if `getByLabel` cannot find the field, a screen reader cannot either. (Where the markup gives us nothing stable, ask the developers for a `data-testid`. That is a reasonable thing to request.)

● Consider next — only the happy path is covered

The most valuable login tests are the ones this file does not have yet: wrong password, unknown user, empty fields, and — importantly — that a failed login does *not* create a session. Those are where the security bugs live. Worth a follow-up PR; you do not need to do it in this one.

The corrected version

```
const { test, expect } = require('@playwright/test');

test.describe('Login', () => {
  test('logs a registered user in and lands them on the dashboard', async ({ page }) => {
    await page.goto('/login'); // baseUrl comes from the config

    await page.getByLabel('Email').fill(process.env.TEST_USER_EMAIL);
    await page.getByLabel('Password').fill(process.env.TEST_USER_PASSWORD);
    await page.getByRole('button', { name: 'Sign in' }).click();

    // Assert what a logged-in user can see, not just where the browser went.
    await expect(page).toHaveURL(/\/dashboard/);
    await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
    await expect(page.getByTestId('user-menu')).toContainText(process.env.TEST_USER_EMAIL);
  });

  test('rejects an incorrect password without creating a session', async ({ page }) => {
    await page.goto('/login');

    await page.getByLabel('Email').fill(process.env.TEST_USER_EMAIL);
    await page.getByLabel('Password').fill('definitely-wrong');
    await page.getByRole('button', { name: 'Sign in' }).click();

    await expect(page.getByRole('alert')).toHaveText('Invalid email or password');
    await expect(page).toHaveURL(/\/login/);

    // A failed login must not leave a usable session behind.
    await page.goto('/dashboard');
    await expect(page).toHaveURL(/\/login/);
  });
});
```

With, in `playwright.config.ts` :

```
export default defineConfig({
  use: {
    baseUrl: process.env.BASE_URL ?? 'http://localhost:3000',
    trace: 'retain-on-failure',      // makes CI failures diagnosable without reproducing
    screenshot: 'only-on-failure',
  },
  retries: process.env.CI ? 2 : 0,
});
```

Summary

PRIORITY	ISSUE	EFFECT IF UNFIXED
●	No assertions — <code>console.log</code> instead	The test can never fail; false confidence
●	Wrong import (<code>playwright</code> not <code>@playwright/test</code>)	The test does not run at all
●	<code>waitForTimeout(5000)</code>	Slow suite, and flaky on loaded CI machines
●	URL-only verification	Passes when the page is broken
●	Manual browser lifecycle	Leaked processes; no cleanup on failure
●	Hardcoded URL and credentials	Cannot run outside one machine; bad habit with real secrets
●	Vague test name	Unreadable CI reports
●	Implementation-coupled selectors	Breaks on refactor; misses accessibility signal
●	Happy path only	The security-relevant cases are untested

The one thing to take away: assertions. Everything else here is craft that comes with practice, but a test that cannot fail is the only issue on this list that actively makes things worse.

Genuinely good first automation — the structure of the flow is right, and that is the part that is hard to teach. Want to pair on the negative cases tomorrow? That is the fun part.

Multi-Currency Settlement with FX Conversion

The shape of this feature

Fifteen currencies, three rate providers with failover, a 60-second rate lock, per-currency minimums, and cross-border regulatory compliance. Every one of those is a place where money can be quietly wrong.

The thing that makes FX settlement different from most features: **there is often no error**. A payment either succeeds or fails, visibly. An FX conversion at a stale rate succeeds, looks completely normal, and is wrong by an amount nobody notices until reconciliation — or until a regulator does. So the strategy leans hard on invariants, reconciliation and property-based checks, not just on example-based tests.

A. Risk-based test strategy

1. Risk register

Scored on impact (financial, regulatory, reputational) and probability, and — critically — **detectability**, because an undetectable defect is more dangerous than a loud one.

● HIGH RISK – EXHAUSTIVE COVERAGE, SHIPS ONLY WITH EXPLICIT SIGN-OFF

#	RISK	WHY IT IS HIGH
H1	Incorrect conversion or rounding — merchant settled the wrong amount	Direct financial loss, at scale, silently. A half-cent error on 100K transactions is real money and a real audit finding.
H2	Currency precision handled wrong — JPY has 0 decimals, KWD/BHD/JOD have 3	Classic and catastrophic: treating ¥1000 as 10.00 is a 100× error. Any hardcoded assumption of two decimals is a defect waiting for a Japanese merchant.
H3	Floating-point arithmetic on money	<code>0.1 + 0.2 !== 0.3</code> . Accumulates across settlement batches, breaks reconciliation, and is invisible in the happy path.
H4	Rate lock not honoured — quoted at one rate, settled at another	Direct financial and legal exposure: the quote is a commitment to the merchant.
H5	Provider failover fails or picks a bad rate	Fallback paths are the least-exercised and most-trusted code in any system.
H6	Double settlement / lost settlement on retry or partial failure	Money created or destroyed. The single worst outcome in payments.
H7	Regulatory breach on cross-border — sanctions screening, reporting thresholds, prohibited corridors	Fines, licence risk. Not recoverable by shipping a patch.
H8	Ledger imbalance — debits and credits do not reconcile after conversion	Undermines every financial report the company produces.
H9	Stale rate accepted as fresh	Looks completely normal; found only by reconciliation, or by a counterparty.

● MEDIUM RISK – AUTOMATED POSITIVE, NEGATIVE AND BOUNDARY COVERAGE

#	RISK
M1	Minimum settlement threshold wrong per currency, or evaluated in the wrong currency
M2	Providers disagree beyond tolerance and no outlier rejection occurs
M3	Provider timeout or partial response handled poorly (hangs, retries a non-idempotent call)
M4	Rate-lock expiry boundary — the request that arrives at exactly 60 seconds
M5	Weekend, holiday and market-close rate behaviour
M6	Timezone and settlement cutoff errors (a settlement dated to the wrong business day)
M7	Refunds and chargebacks in a converted currency — at which rate?
M8	Fee calculation order relative to conversion (convert-then-fee $\neq$ fee-then-convert)
M9	Audit trail incomplete — which rate, from which provider, at what time
M10	Concurrency: simultaneous settlements for one merchant across currencies

● LOW RISK – SMOKE TESTS AND EXPLORATORY

#	RISK
L1	Currency symbol, formatting and locale display in the dashboard
L2	Sorting and filtering multi-currency lists
L3	Export formatting (CSV/PDF) of converted amounts
L4	Historical rate display in reports

2. Testing approach per risk level

LEVEL	COVERAGE REQUIRED	TECHNIQUES	GATE
● High	Every path, every currency, every failure mode	Example-based + property-based + fault injection + reconciliation invariants + manual finance review of golden values	Blocks release; sign-off from QA, finance, and compliance for H7
● Medium	Positive, negative, boundary	Boundary value analysis, state transition, contract tests, controlled-clock tests	Blocks release on failure
● Low	Happy path plus exploratory	Smoke automation, visual review, exploratory charters	Does not block; tracked

Why property-based testing earns its place at the High level. For conversion and rounding, example-based tests only prove the examples someone thought of. Properties hold for *all* inputs, and a generator finds the case nobody imagined:

- Round-tripping a conversion returns to within one minor unit of the original.
- The sum of split settlements equals the single settlement of the total (no rounding leakage across batches).
- Converting through an intermediate currency agrees with the direct rate within tolerance.
- No settlement result ever has more decimal places than the target currency permits.
- Total debits equal total credits, in every currency, after every operation.

That last one is the invariant I would run continuously in production, not only in tests.

3. Test coverage detail for the High risks

H1/H2/H3 — conversion, precision, arithmetic. A golden-value matrix agreed with finance: 15 currencies × representative amounts (minimum, typical, maximum, and values chosen to land exactly on a rounding boundary), with expected results calculated independently and reviewed. Plus static analysis banning float types on monetary values — this should be a lint rule and a code review standard, not only a test.

Specific cases that must exist:

CASE	WHY
JPY (0 decimals): ¥1,000 → USD and back	Zero-decimal handling
KWD (3 decimals): 1.001 KWD → EUR	Three-decimal handling; most systems assume two
Rounding exactly at .005 in both directions	Half-up vs half-even (banker's rounding) — and <i>who absorbs</i> the difference must be a documented product decision, not an implementation accident
Very large amounts near the ledger's maximum	Overflow and precision loss
Very small amounts converting to zero	A settlement of 0 must be rejected, not silently succeed
An amount that rounds <i>below</i> the minimum after conversion	Ordering of validation against conversion

H4 — the 60-second rate lock. The system needs an injectable clock; otherwise these tests are 60-second sleeps and nobody will run them. With a controllable clock:

- Settle at $t=0$, $t=30$, $t=59.9$ → locked rate applied.
- Settle at $t=60.1$ → rejected or re-quoted, per the product rule, never silently re-rated.
- **$t=60.0$ exactly** → the behaviour must be defined in writing before it is tested.
- Rate changes materially during the lock → the locked rate still wins.
- Two concurrent settlements against the same quote → one succeeds (idempotency).
- Clock skew between services → the lock is evaluated against one authoritative time source.
- A quote never used simply expires and is cleaned up.

H5 — provider failover. A mock proxy in front of the three providers, able to inject: timeout, 500, 429, malformed JSON, a valid-but-absurd rate (a decimal point moved), a stale timestamp, and a slow-but-successful response. Cases: primary fails → secondary used; primary and secondary fail → tertiary; all three fail → **settlement is refused, not completed at a guessed rate**; primary recovers → traffic returns; and the sanity check that matters most, a rate outside tolerance of the others is rejected rather than used. Circuit-breaker state transitions get their own tests.

H6 — double settlement. Idempotency keys on every settlement request; retry the same request 100 times concurrently and assert exactly one settlement exists. Kill the service mid-settlement (after the external call, before the local commit) and assert recovery produces one

settlement, not zero and not two. This class of bug is found by fault injection, never by functional testing.

H7 — regulatory compliance. Test data covering sanctioned jurisdictions, amounts above and below reporting thresholds, prohibited corridors, and missing beneficiary information. Every decision must be logged immutably with the rate, provider and timestamp. This is the one area where I would insist on external validation: QA proves the controls work as specified; compliance owns whether the specification is right. Those are different jobs and conflating them is how companies get fined.

H8/H9 — invariants and staleness. Continuous reconciliation as a test *and* as a production monitor: after every settlement run, debits equal credits per currency; the sum of converted amounts matches the source total within documented tolerance; every settled amount traces to a specific rate, provider and quote. Rates carry an age; anything beyond the freshness threshold is rejected, and that threshold is tested at its boundary.

4. Environments needed

ENVIRONMENT	PURPOSE	FX PROVIDERS	DATA
Local	Developer feedback in seconds	Fully mocked, fixed rate matrix	Seeded fixtures, all 15 currencies
CI (ephemeral)	The merge gate	Deterministic mocks	Generated per run
FX chaos	Failover, latency, malformed responses, provider outages	Fault-injecting proxy	Same fixtures, faults on demand
Staging	Integration truth	Provider sandboxes , real protocols	Anonymised production-shaped data
Compliance / pre-prod	Regulatory validation, audit rehearsal	Sandbox, full audit logging on	Regulatory scenario set; access-restricted
Production	Post-deploy confidence	Real	Synthetic merchant, minimum-value settlements, plus continuous reconciliation monitors

Two of these are non-obvious and I would fight for both. **The chaos environment**, because failover paths are the least-tested and most-trusted code in the system, and the only way to

know they work is to break things on purpose. **The compliance environment**, because rehearsing an audit before a regulator asks is much cheaper than discovering during one that the audit trail has a gap.

A note on production: for a feature like this, testing in production is not a compromise — it is the only place where real rates, real provider behaviour and real volume exist together. It just has to be done with synthetic merchants, small amounts, and monitors that catch a problem in minutes.

B. Test data strategy

1. What test data is needed

CATEGORY	CONTENT
Currency configuration	All 15: ISO 4217 code, decimal places (0/2/3), minimum settlement, supported corridors, cutoff time, regulatory flags
Rate fixtures	Normal rates; extreme but real (a 20% move); crossed/inverted pairs; stale timestamps; missing pairs; provider-disagreement sets; absurd outliers
Merchant profiles	Per jurisdiction and settlement currency; single- and multi-currency; sanctioned-country merchant; a merchant below every minimum
Transaction sets	Amounts below, exactly at, and above minimum; zero-decimal currencies; three-decimal currencies; amounts landing on rounding boundaries; maximum-value; multi-currency batches
Regulatory scenarios	Above and below reporting thresholds; prohibited corridors; incomplete beneficiary data; high-risk jurisdictions
Temporal data	Weekend, holiday, market close, month end, and a leap day; rate-lock boundaries at 59/60/61 seconds
Historical rate series	For reproducing a specific past settlement exactly — required for audit rehearsal and for defect reproduction

2. Managing currency-specific data

One source of truth, and it is not the tests. The currency matrix — decimals, minimums, cutoffs — is configuration the *service* already owns. Tests read the same configuration rather than duplicating it. Duplicating it guarantees the day where JPY's minimum changes in production, the tests keep passing against the old value, and QA is the last to know.

Builders, not fixtures files. `aSettlement().inCurrency("JPY").atMinimum().build()` reads clearly and stays correct when the minimum changes, because the builder resolves it from configuration. Hand-maintained JSON fixtures with hardcoded amounts rot within a quarter.

Data-driven across all 15 currencies by default. Every conversion test runs as a parameterised case over the full currency matrix — the JPY and KWD bugs are found by running the same test 15 times, not by someone remembering to write a JPY case. Adding a sixteenth currency then means adding a config row, and any test that does not handle it fails immediately. That is exactly the feedback you want.

Golden values reviewed by finance. The expected results for the conversion matrix are calculated independently — by finance, in a spreadsheet, once — and committed as golden files. If QA derives expected values from the same code under test, the test proves only that the code is consistent with itself. Any change to a golden value requires review from finance, which is a feature, not friction.

Anonymised, never copied. No production merchant or beneficiary data in any lower environment. Production-*shaped* data, synthetically generated: same distributions, volumes and edge cases, no real people.

3. FX rate testing — mocking versus real

Neither alone works. Mocks are deterministic but can drift from reality; real providers are realistic but make assertions impossible, since the rate changes between the arrange and the assert. The answer is layered, and each layer is chosen for what it can actually prove.

LAYER	RATES	WHAT IT PROVES	RUNS
Unit / service	Hardcoded deterministic matrix	Conversion maths, rounding, precision — exactly assertable	Every commit
Contract	Provider sandbox, schema-validated	Their API still returns what we parse; catches <i>their</i> breaking changes	Nightly; alerts, does not block merges
Failover / chaos	Fault-injecting proxy	Timeouts, malformed payloads, outliers, all-providers-down	Every merge to main
Integration (staging)	Recorded-and-replayed real rates (VCR-style)	Realistic values with deterministic assertions	Every merge to main
Exploratory	Live sandbox	Behaviour nobody predicted	Per release
Production canary	Real, live	The whole thing genuinely works end to end	Continuous, synthetic merchant

The default is mocked. Any test asserting a specific settlement amount must control the rate — otherwise it is asserting against a moving target, and a test that fails because the market moved is a test the team will disable within a fortnight.

Record-and-replay is the pragmatic middle. Capture real provider responses once, replay them deterministically, refresh the recordings weekly. Realistic payloads, stable assertions. The refresh is what stops the recordings from silently diverging from what the provider actually sends today.

Contract tests are non-negotiable with three providers. Three external APIs are three independent sources of breaking change on somebody else's release schedule. Nightly schema validation against each sandbox catches a field rename before it reaches production. It alerts rather than blocking, because a provider's sandbox being down should not stop our team from merging.

Time is mocked, not slept through. The 60-second lock is tested with an injectable clock. One real-time end-to-end test at the boundary is worth keeping as a sanity check that the clock abstraction has not diverged from wall-clock reality — but only one.

And in production, verify rather than assume. A synthetic merchant settling minimum amounts across all 15 currencies daily, with automatic reconciliation against an independent rate source. That is the check that catches what every environment above it mocked away.

Closing note

Two threads run through all three parts of this document.

Evidence over assertion. The 30-day plan starts with ten days of measurement rather than ten days of test-writing. The automation section reports eleven real defects found in a demo application, with reproduction steps, rather than claiming coverage. The metrics section names the numbers I would refuse to report, not only the ones I would.

Quality is a property of the system, not of the QA function. Every proposal here — shift-left refinement, developer-owned unit tests, mentoring juniors into automation, production monitoring as a first-class test environment — moves quality *toward* the team and away from a person at the end of the pipeline whose job is to catch things. With five engineers deploying daily, any strategy that depends on QA being the safety net fails on contact with the second sprint.

Test Plan

Application under test: <https://demoqa.com/books> (web) · <https://demoqa.com/swagger> (API) **Author:** Senior QA Engineer candidate · **Date:** 25 Aug 2025 **Status:** Executed — results in Appendix B

1. What we are testing and why

The Book Store lets a person register, sign in, browse a catalogue of books, keep a personal collection, and sign out. The product's value sits in one place: **a user's collection is correct and private**. Everything else — layout, sorting, pagination — is secondary to that.

So the plan is organised around the question *"can the collection be wrong, lost, or seen by the wrong person?"* rather than around a list of screens.

In scope

AREA	WEB	API
Registration and password policy	✓ (form contract)	✓
Authentication and session handling	✓	✓
Catalogue browse, search, sort, paginate	✓	✓
Book detail	✓	✓
Collection: add, list, replace, delete, clear	✓	✓
Authorisation between users	—	✓
Cross-layer consistency (UI state = API state)	✓	✓

Out of scope, and why

- **Load and stress testing** — we do not own the shared demo environment; a load run would degrade it for everyone else using the site.

- **Penetration testing** — beyond the authorisation checks that need no special permission.
 - **Visual regression** — worth adding once the DOM stabilises, but it is not the first thing that protects a collection from corruption.
 - **Solving reCAPTCHA** — see *Known constraints*.
-

2. Risk analysis — what drives coverage

Coverage follows risk, not screen count. Probability and impact are scored from how the system actually behaves, which is why several rows below already read "Confirmed" (see Appendix B).

#	RISK	IMPACT	PROB.	PRIORITY	HOW IT IS COVERED
R1	One user reads or modifies another user's collection	Critical	Low	P0	@security scenarios: cross-token read, write, delete
R2	A book is added but not persisted (the UI lies)	High	Medium	P0	Cross-layer assertion: UI action, then API read-back
R3	Delete removes the wrong book, or does not persist	High	Medium	P0	Delete by ISBN, then API read-back; plus a cancel-path test
R4	Session does not end on logout	High	Low	P0	Post-logout access to /profile must be refused
R5	Failed authentication reported as success	High	Confirmed	P0	Negative auth scenarios (found BUG-006)
R6	Weak passwords accepted	Medium	Low	P1	Policy table plus an 8-character boundary scenario
R7	Duplicate add corrupts the collection	Medium	Medium	P1	Duplicate add → 1210, then state re-verified
R8	Concurrent sessions evict each other	Medium	Confirmed	P1	Documented as BUG-009 ; drove the suite's token design
R9	Search returns wrong or no results	Medium	Medium	P1	Exact, partial, no-match and case searches
R10	Catalogue payload changes shape	Medium	Medium	P1	Schema assertion on every catalogue entry
R11	Pagination or sorting wrong	Low	Medium	P2	Exploratory plus one automated check (found BUG-004)

#	RISK	IMPACT	PROB.	PRIORITY	HOW IT IS COVERED
R12	Cosmetic and responsive defects	Low	High	P2	Exploratory each release

The rule this produces: P0 is automated and gates every deployment. P1 is automated and runs on every pull request. P2 is exploratory, with selective automation once a defect is found twice.

3. Test approach by layer

The same journey is covered at two layers, deliberately, because they answer different questions.

few	E2E (Playwright) register → ... → logout 1 journey + critical negatives	does the journey hold together for a person?
some	API (Karate) contracts, auth, error codes 23 scenarios	is the behaviour correct and complete?
many	Unit tests (owned by developers)	is the logic right?

Why so much sits at the API layer: that is where the rules live (password policy, authorisation, error codes), it runs roughly twenty times faster, and it does not break when a class name changes. Every rule that *can* be tested there *is* tested there. The UI layer then only has to prove the journey works end to end — one E2E test, not thirty.

Test design techniques applied

TECHNIQUE	WHERE
Equivalence partitioning	Password classes; ISBN valid / unknown / malformed
Boundary value analysis	Password at exactly 8 characters, and at 7
State transition	Collection: empty → one book → duplicate rejected → empty
Decision table	Auth outcomes: user exists × password correct × token present
Error guessing	Deleted account holding a live token; token replaced mid-session
Contract / schema testing	Every catalogue and collection payload

4. Test coverage

4.1 API scenarios — automated in Karate (23 scenarios, all passing)

ENDPOINT	POSITIVE	NEGATIVE AND EDGE
POST /Account/v1/User	201, empty collection, well-formed body	7 password-policy violations → 400/1300; duplicate user → 406/1204; 8-char boundary → 201
POST /Account/v1/GenerateToken	200, Success, token issued	Wrong password → 200 + Failed (BUG-006)
POST /Account/v1/Authorized	true for valid credentials	Wrong password → 404/1207 (BUG-007)
GET /Account/v1/User/{id}	200, collection matches	No token → 401/1200; malformed token → 401; another user's id → 401; deleted account → 401/1207 (BUG-008)
DELETE /Account/v1/User/{id}	204, and the token dies with it	Another user's account → 401
GET /BookStore/v1/Books	200, schema, ISBNs unique	—
GET /BookStore/v1/Book?ISBN=	200, full schema	Unknown ISBN → 400/1205
POST /BookStore/v1/Books	201 for one book and for several	Duplicate → 400/1210 with state unchanged; no token → 401; another user's id → 401
PUT /BookStore/v1/Books/{ISBN}	200, old ISBN gone, new one present	—
DELETE /BookStore/v1/Book	204, removed	Book not owned → 400/1206
DELETE /BookStore/v1/Books?UserId=	204, collection emptied	—

4.2 Web scenarios — automated in Playwright (7 tests, all passing)

ID	SCENARIO	TAG
W-01	Full journey: register → login → search → add → view → delete → logout	@e2e
W-02	Cancelling the delete dialog keeps the book (state seeded via API)	@regression
W-03	Unknown credentials → error shown, no session created	@smoke
W-04	Valid user, wrong password → rejected	@smoke
W-05	/profile refuses anonymous access	@smoke
W-06	Registration form contract and reCAPTCHA present	@smoke
W-07	Password policy enforced (via API — the rule the form fronts)	@smoke

W-01 additionally asserts that after each UI action the **API agrees** with what the screen shows. That is the check which catches "the UI said it saved, and it didn't".

4.3 Manual and exploratory — each release, time-boxed to 90 minutes

Charters, not scripts:

- Catalogue sorting and pagination applied on top of a filtered search.
- Search behaviour: partial words, case, punctuation, ISBN, and the no-match empty state.
- Session: browser back after logout; two tabs; token expiry at six hours.
- Responsive layout at 360 px, 768 px and 1440 px.
- Keyboard-only journey and screen-reader labels — duplicate ids are already a known problem here (BUG-001, BUG-002).

5. Test data strategy

NEED	APPROACH
User accounts	Generated per test (<code>qa_<uuid></code>), created via API, deleted in teardown
Passwords	Meet the documented policy; violation cases live in a data table
Books	The seeded catalogue is read-only and stable; ISBNs referenced by constant
Collections	Seeded through the API whenever the test is not <i>about</i> adding a book

Nothing is shared between tests. The demo environment's user namespace is global and shared with strangers on the internet, so any fixed username eventually collides with `406 User exists!`. Isolation is what makes the suite parallel-safe, and the same principle applies to any shared staging environment.

6. Environments

ENVIRONMENT	PURPOSE	DATA	SUITE
Local	Developer feedback	Ephemeral	<code>@smoke</code> on pre-push
CI (pull request)	The merge gate	Generated per run	<code>@smoke</code> + <code>@e2e</code> , chromium
Staging	Release candidate	Anonymised, refreshed nightly	Full regression, three browsers
Production	Post-deploy confidence	Read-only synthetic account	Smoke only, non-destructive

`BASE_URL` (Playwright) and `karate.env` (Karate) are the only things that change between them. There are no forks of the test code per environment.

7. Entry and exit criteria

Entry: the build deploys; the API answers a health check; test accounts can be created; the feature has acceptance criteria written down.

Exit:

- All P0 and P1 automated tests pass.
 - No open Critical or High defect in the flows under test.
 - Exploratory charters completed and findings triaged.
 - Every test disabled during the cycle has a ticket and a named owner. A quarantined test with no owner is deleted coverage that nobody has admitted to.
-

8. Reporting

- **Per run:** Playwright HTML report with traces, and the Karate HTML summary, both published as CI artefacts — a failure should be diagnosable from the build alone, without anyone re-running it locally.
- **Per release:** one page. What was covered, what was found, what is still risky.
- **Defects:** logged with steps, expected versus actual, evidence and environment. Appendix B shows the format used here.

9. Known constraints

Registration is behind Google reCAPTCHA. It cannot be driven from an automated browser, and defeating it would only work against this demo site — a test that passes by cheating gives false confidence. Two consequences:

1. Accounts for UI tests are provisioned through `POST /Account/v1/User`, which produces exactly the account a real registration would.
2. The UI suite asserts the *form's* contract — fields present, reachable from login, captcha enforced — and the registration rules themselves are covered at the API layer.

On a product I owned, the fix would be a reCAPTCHA test key in non-production environments; Google publishes one for this exact purpose. That would be my first request to the development team.

Single active token per user. Found while building the suite (BUG-009): calling `GenerateToken` invalidates any existing session. Mixed UI and API tests therefore reuse the browser's own session token rather than minting a competing one.

Third-party ads intercept clicks. The suite removes ad containers after navigation. On a product I owned this would be environment configuration instead — tests should not have to

fight the page they are testing.

Defects Found

Every item below was reproduced against the live application on 2025-08-25 while writing the suites. Each one is either encoded in a test or explains why a test is written the way it is.

Web UI

BUG-001 — Three different buttons share `id="submit"` on `/profile`

Severity: Medium (accessibility + automatability) **Steps:** Log in → open `/profile` → inspect the DOM. **Actual:** *Logout*, *Delete Account* and *Delete All Books* all render `id="submit"`. **Expected:** Ids are unique per document (HTML spec). **Why it matters:** Duplicate ids break `document.getElementById`, break screen-reader label association, and force every automated test to fall back to text matching. It is a one-line fix that removes a whole class of future flakiness.

BUG-002 — Two buttons share `id="addNewRecordButton"` on the book detail page

Severity: Medium **Actual:** *Back To Book Store* and *Add To Your Collection* carry the same id, so `#addNewRecordButton` resolves to whichever comes first in the DOM — a test written against it would silently click the wrong control.

BUG-003 — "Delete All Books" is rendered twice on `/profile`

Severity: Low **Actual:** The profile page renders two identical *Delete All Books* buttons. **Expected:** One. A duplicated destructive action is a genuine usability risk.

BUG-004 — Empty collection paginates as "Page 1 of 0"

Severity: Low **Steps:** Log in with an empty collection, or delete the last book. **Actual:** The pager reads `Page 1 of 0`. **Expected:** `Page 0 of 0`, or the pager is hidden when there is nothing to page.

BUG-005 — `/profile` briefly renders its logged-out panel after a successful login

Severity: Low (perceived quality), High (as a source of test flakiness) **Steps:** Submit valid credentials on `/login`. **Actual:** The browser lands on `/profile` and, for a few hundred milliseconds, shows the "Login in Book Store" panel before the authenticated view replaces it. **Impact:** Any test — or user — that reacts to the first paint sees the wrong state. The suite works around it by waiting on `#userName-value`, not on the URL.

OBS-001 — Logout is labelled "Logout" on /profile and "Log out" on the book detail page

Cosmetic inconsistency; the suite matches `/log ?out/i` rather than either literal.

API

BUG-006 — Failed authentication returns `200 OK`

Severity: High (contract correctness) **Request:** `POST /Account/v1/GenerateToken` with a valid username and a wrong password. **Actual:** `200 OK` with

```
{"token":null,"expires":null,"status":"Failed","result":"User authorization failed."}
```

Expected: `401 Unauthorized`. **Why it matters:** Every generated client, gateway, retry policy and alerting rule keys off the status code. A failed credential check that reports success is invisible to monitoring, and a client that only checks `res.ok` will happily proceed with a null token.

BUG-007 — Wrong password reported as "User not found"

Severity: Medium **Request:** `POST /Account/v1/Authorized` with an existing username and a wrong password. **Actual:** `404` with `{"code":"1207","message":"User not found!"}`

Expected: `401` with a credential-failure code. The user exists; the password does not match. Reporting `404` misleads clients and support alike.

BUG-008 — HTTP status and error code disagree

Severity: Medium **Request:** `GET /Account/v1/User/{id}` for a deleted account, with that account's token. **Actual:** HTTP `401` carrying `{"code":"1207","message":"User not found!"}` — the same error code that `/Authorized` pairs with HTTP `404`. **Expected:** One code, one status, consistently. As it stands a client cannot map error codes to statuses reliably.

BUG-009 — Generating a token silently invalidates the existing session

Severity: High (undocumented behaviour with real product impact) **Steps:** Log in through the UI, then call `POST /Account/v1/GenerateToken` for the same user. Every subsequent request from the browser session returns `401`. **Actual:** The back end keeps one valid token per user; a new token kills the old one. **Expected:** Either concurrent sessions are supported (a user on a phone and a laptop is not an edge case), or the single-session rule is documented and the evicted session is told why. This is not visible in the Swagger contract at all. **Impact on the suite:** cross-layer assertions reuse the browser's own session cookie instead of minting a second token — see `sessionTokenFromBrowser()`.

OBS-002 — `/Account/v1/Authorized` returns a bare JSON primitive

Responds with `true` rather than an object such as `{"authorized": true}`. Legal JSON, but awkward for typed clients and impossible to extend without a breaking change.

PASS — Object-level authorisation is correctly enforced

Worth recording as a *positive* result, because it is the flaw this API's shape invites: the `userId` travels in the request body, so a naive implementation would trust it. It does not. With user A's token:

- `GET /Account/v1/User/{B's id} → 401`
- `POST /BookStore/v1/Books` with `userId = B` → `401`
- `DELETE /Account/v1/User/{B's id} → 401`

Covered by the `@security` scenarios in `account.feature`.

Environment constraint (not a defect)

Registration is protected by Google reCAPTCHA. It cannot be automated through the browser, and defeating it would only work against this demo site. Registration is therefore performed through the public API for test-data purposes, and the UI suite asserts the form's contract instead. In a real product the fix is a test-environment captcha bypass key — reCAPTCHA ships one for exactly this reason.

SET IN IBM PLEX SUITES EXECUTED 25 AUG 2025 SOURCE IN THE ACCOMPANYING GIT REPOSITORY